

Evolutionary Optimization of Baba Is You Agents

Anonymous Author
Anonymous University
City, County
Email

Anonymous Author
Anonymous University
City, County
Email

Anonymous Author
Anonymous University
City, County
Email

Abstract—Baba is You is a challenging puzzle game in which the player can modify the rules of the game. This yields a large variety of puzzles and an enormous state space to be searched through. Recently, the Feature Space Search algorithm has shown great results in Sokoban, which apart from the rule modification shares many similarities to Baba is You. It uses multiple heuristics to guide the search into promising regions of the search space. In this work, we are proposing a similar concept for solving Baba is You based on multiple heuristics that are aggregated for guiding a tree-based search process. However, finding parameters for weighting/prioritizing the different heuristics is a non-trivial task. This process is done, by applying evolutionary algorithms for single- and multi-objective optimization. Specifically, we compare the effects of these different optimization schemes on the agents’ general level-solving capabilities. In all cases, the agent was able to adapt well to the training and test set with no significant differences among the optimization schemes. Compared with state-of-the-art Baba is You agents our search-based approach shows an improved performance in terms of the number of levels being solved, as well as a reduction in the average time required to solve a level.

Index Terms—Baba Is You, Parameter Optimization, Evolutionary Algorithms

I. INTRODUCTION

Sokoban is a puzzle game in which the player controls a character that pushes boxes around in a warehouse, trying to move them to specific target locations. The player can only push one box at a time, and the boxes cannot be pulled. Sokoban and similar puzzle games present players with many challenging levels to solve. Finding a solution for the more complex levels can be a tough challenge for humans and AI agents alike. Their large state space as well as the long and precise action sequences required to solve a level are hard to overcome while developing AI agents [1]. Sokoban has been proven to be computationally difficult and belongs to the class of NP-hard problems [2].

In this work, we focus on the Sokoban-like game Baba is You (see detailed description in section II). The game features the unique concept of allowing players to modify the underlying rules over the course of a single level. We developed a search-based agent for solving Baba is You levels making use of multiple heuristics to guide its search process. Each of these heuristics concentrates on another aspect of the game’s state and action space and has shown its use in solving a subset of levels. For developing an agent that aims to solve as many levels as possible, we combine these heuristics into a single scoring function. However, weighting these heuristics

has shown to be a non-trivial task. Therefore, we made use of several optimization schemes to tune the heuristics’ weights.

In this study, we develop a Baba is You agent capable of solving multiple levels efficiently. Our contributions can be summarized by:

- **Creating heuristics for solving Baba is You levels:** We implemented several heuristics taking various game-state measurements into account. These heuristics have been developed based on our own insights and analysis from game-play experience and will be used to guide the tree-based search process of our agent.
- **Developing a performant Baba is You agent:** Solving levels needs to be quick and accurate. In our work, we aggregate several heuristics into a single scoring function. To reduce the agent’s computation time, we allow for heuristics to be permanently removed while aiming to keep the agent’s success rate high and the number of required iterations low.
- **Comparisons to state-of-the-art agents:** The proposed agent’s parameters are tuned using several evolutionary optimization algorithms. Our final agent is tested against available submissions of the Keke AI competition.

In Section II, we describe the game Baba is You and the Keke AI Framework which will be used during the remainder of our work. Section III summarizes recent works on Sokoban-like games on which our proposed method will be based. We present our agent and accompanying heuristics in Section IV. The process of optimizing the agent’s parameters is described in the subsequent Section V. The resulting agent is compared to the state-of-the-art in Section VI. Finally, in Section VII, we summarize our work and ideas for how it can be further improved in the future.

II. BABA IS YOU AND THE KEKE AI FRAMEWORK

Baba Is You is a puzzle video game in which the player walks around a grid-like game board to reach a given goal. It shares many resemblances with Sokoban-like puzzle games, with the main difference that the player is able to change the rules of the game and alter the behavior of the player character and other in-game elements. The game has received widespread acclaim for its innovative gameplay and has won numerous awards.

To better explain the unique concept of the game, we describe the rule modification in more detail. For this purpose, the game consists of a series of levels, each of which presents

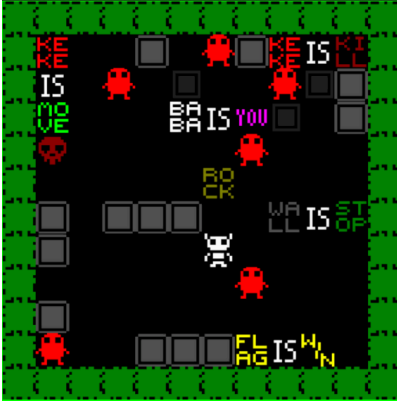


Fig. 1: An exemplary level from the Keke is You AI framework showing multiple objects and rules.

the player with a game board filled with various words written on tiles representing either objects of the game’s world (e.g. Rock, Keke, Flag), concepts (e.g., win or die), or the linking word “is”. Each object word is linked to tiles showing a graphical representation of this word. Multiple words can be arranged to represent different rules or properties that can be applied to the game, such as “Keke is You” or “Rock is Push.” Rules consist of a combination of three words. The middle word has to be *Is* to form a valid rule. Therefore rules can be subdivided into a prefix and a suffix. The prefix is mostly described by an object part like *Baba Is*, *Flag Is*. The suffix can result in a feature or action, e.g. *Is Win*, *Is You*, *Is Kill*. A classic rule set consists of *Baba Is You* and *Flag Is Win*. Here, the player controls the object “Baba” (a little dog) and touching the flag represents the winning condition. While moving around, the player can rearrange the tiles to modify or invalidate existing rules and create entirely new combinations until the current winning condition is satisfied.

In terms of AI development, *Baba Is You* represents a challenging problem for search-based approaches. Whereas Sokoban-like games are already known for their large search space, the interactive modification of rules results in even more possibilities. Furthermore, as in many other puzzle games, a single wrong decision can make the puzzle unsolvable, limiting the use of limited-depth search-based algorithms such as the Rolling Horizon Evolutionary Algorithm [3].

To study the development of AI agents, the Keke AI Competition has been created. The associated framework allows the development of agents using javascript aiming to solve a given set of levels. Therefore, each agent receives the level’s starting configuration, and a simulator allowing one to anticipate the result of its actions. The computation time of each agent is limited to a maximum time and a maximum number of iterations. Being part of the IEEE Conference on Games 2022, the competition provided a set of baseline agents and attracted multiple entries during its first year. Given a budget of 10 seconds and 10000 iterations, the best agent was able to achieve a 66% win rate on a hidden set of levels according to the leaderboard on the competition webpage.

III. RELATED WORK

Since *Baba is You* is a relatively new problem in the AI domain, we mainly focused on reviewing related work of Sokoban-like games. Sokoban has a long tradition for being targeted by AI agents [4] and still remains a tough problem due to its long action sequences required to solve some levels. This is a major challenge for search-based approaches since with every move, the search space grows exponentially.

Luckily there often exists not just a single correct action sequence, but multiple sequences to solve a level. For example, in an open room without any obstacles, the agent may take many paths to reach a given destination. On the contrary, there also exist choice points in which a single bad decision could leave the level unsolvable. This is the case when a pushable block is moved into a corner and thus cannot be moved anymore.

There are two types of solving Sokoban-like games. While some Sokoban solvers rely on brute force (exploring many states with low computational costs in between) others make use of heuristics that tend to steer the agent in the right direction at the cost of some computational overhead.

Furthermore, both of these techniques can benefit from integrating expert knowledge that can be used to reduce the search space by decomposing the original problem into several simpler sub-problems. In the context of Sokoban, this has been done by splitting the level into rooms and tunnels [5]. Sadly, effective expert rules for reducing the complexity of *Baba is You* levels are not yet known.

More generally, abstractions can be used to reduce the complexity of the action and state space [6] by e.g. clustering similar states. Not all of these techniques ensure completeness (every solvable level can be used) after the abstraction has been applied, but they have shown to benefit search-based approaches in other complex domains [7].

Apart from these exhaustive search agents, other agents have been developed in which the agent mixes the planning and execution process. Here, the available time for the search process is limited and after exhausting the computational budget an action needs to be applied before the agent can continue planning given the new state observation [8]. Alternatively, the search may be reduced to a sampling-based approach such as Monte Carlo Tree Search [9].

In the context of Sokoban, the Feature Space Search (FESS) algorithm [10] has been shown to perform well in comparison to other search-based agents. It has been the first algorithm to solve 90 benchmark problems given a budget of 10 minutes per problem. This has been achieved by using multiple heuristics.

In contrast, the development of *Baba is You* AI agents is still in its infancy. Simple exhaustive search agents such as depth- and breadth-first search are provided as baseline agents in the Keke AI framework. Additionally, it included a single-heuristic agent that applies a best-first-search algorithm and has been used as a solver for the *Baba is Y’all* level editor [11]. Developing our own agent, we start with developing multiple heuristics to be aggregated into a single heuristic score for guiding our search-based agent.

IV. METHODOLOGY

We present a method for solving levels in Baba is You. The agent stores game states in a search tree and uses a priority queue to determine the next node to be expanded. Each node contains the following information:

- The score of the node (lower is better)
- The map-state saved as an ASCII representation
- The step sequence as a list
- The parent of the node
- Whether it is a final game state and its respective result
- Whether all player objects have died or not

The priority queue consists of all nodes with unexplored child nodes. The nodes in the queue are sorted by their score, with lower scores being prioritized. At each iteration, the frontmost node in the queue is selected and its child states are added to the tree and the queue. For each of the five available actions (moving in 4 directions and waiting) we make use of the simulator to anticipate the outcome of the next action and calculate the respective information to create a new child node, i.e., checking for victory or defeat, calculating a score for the child node based on various heuristics, and adding the child to the queue if it represents a unique game state. To prevent the agent from getting stuck in a loop, we keep track of previously visited game states and only add a child node to the queue if its game state has not been encountered before. For this purpose, a list of all previously visited map states is maintained and compared with the map state of the child node. Once a winning node has been identified, we return the respective action sequence. Contrarily, if no more player objects exist, because either they have died due to an effect or no more rule of the type "<x>is you" exists, the child will be discarded and continued with the next child or the next iteration. After the new nodes are added, the next iteration is started. This process is repeated until either a solution is found or one of the competition constraints is exceeded (max 10.000 iterations or max 10 seconds for a winning solution).

A. Heuristics

The calculation of the heuristic score is the most complex step. In our agent, we make use of multiple heuristics that are weighted and aggregated to generate a single score. These heuristics focus on various aspects of the game and are divided into three categories: (1) object type min-/maximizers, (2) distance min-/maximizers, and (3) meta-heuristics. To allow for fine-tuning of the decision-making process, each heuristic was assigned an adjustable weight that determines its influence on the agent's actions. We also implemented a threshold for these weights to enable the optimization algorithms to disable heuristics if necessary. If a heuristic's weight falls within the threshold, it will return a value of zero without consuming any additional computational resources.

1) *Object type min-/maximizers*: The following heuristics count the number of objects in a given category. The idea was that generating helpful objects can be encouraged and harmful objects can be penalized.

Goals: Counts the number of goal objects, i.e., the objects where our agent would win if it manages to reach them. The larger the number of objects, the larger, or smaller (if the optimizer chooses a negative weight), the score will be. This value is calculated using a logarithmic function to encourage the agent to prioritize the first generated goals more highly and subsequent goals less so that it does not solely focus on increasing the goal count even if it has already reached one.

Playable objects: Counts the number of objects that the agent can directly control and multiplies this value by the weight provided by the optimizer. Given a positive weight, this heuristic encourages the agent to keep as many of the player objects alive and to create new player objects (if possible).

Pushable objects: All objects that the agent is allowed to push are counted here. By default, all objects are immovable and can only be pushed if there is a push rule associated with the object in the currently active level.

Automovers: Auto movers are objects that move a field once every iteration. They also tend to be objects that can kill the player. The number of auto movers is counted linearly and multiplied by the specified weight.

Threats: Threats in the game are simply the objects that can kill the player. This heuristic penalizes/rewards the number of threats linearly.

Sinker objects: Objects that move into "sinks" are destroyed while also removing the sinker object. This means that it is an obstruction for the agent, but it can also be used to destroy other objects and open up new paths to the agent. This heuristic counts the number of sinkers linearly, allowing the optimizer to either punish or reward the creation/destruction of sinker objects.

Stopping objects: Objects can become stopping objects when their object name is combined with the rule "is stop". They prevent all other objects from running through it. They are counted linearly and rewarded/penalized according to their given weight and their amount.

2) *Distance min-/maximizers*: The following heuristics are typically used to minimize distances to specific object types, e.g., goals. Since their weight could be both negative, they could also be considered to be distance maximizers. In practice, however, it has been shown that the optimizers have always chosen minimization, which is why we will speak of minimization in the following for the sake of simplicity. The distances are calculated by averaging the Manhattan distance (the number of moves it takes to reach the object) of all player objects to all objects of a given type.

Minimization of the distance to threats: It may seem counter-intuitive, but minimizing the distance between the agent and killing objects has proven effective in testing. This is because threat objects are often guarding important targets, so it makes sense to search in their vicinity.

Minimization of the distance to Points of Interest (POI): This is one of the most important heuristics, as it makes the agent move near interactable objects. The POIs are goals, movable objects, and words, all of which are weighted differently. This heuristic has an additional fourth weight that punishes

the agent for not having any players left and is intended to counter the fact that killing all player objects naturally causes the distance to all POI objects to be zero.

Minimization of the distance to a winning word when there is only one: Inspired by the macro moves from the FESS paper [10], one of our ideas was to build heuristics that only operate in certain situations. This heuristic is only applied when there is only one "win" word in the level and no victory objects exist yet. Thereby, it helps to guide the agent towards the vicinity of the single "win" word in order to create a victory condition.

3) Meta-heuristics:

Connectivity: We derived the connectivity heuristic used in FESS [10]. Here, the number of closed rooms, i.e., rooms from which one cannot reach other rooms, is counted and penalized, leading the agent to open up as many rooms as possible. This heuristic is computationally expensive, but it has led to the solving of significantly more levels than without it.

Out-of-plan: As with the connectivity heuristic, this heuristic is also derived from the FESS paper [10] but adapted for Baba Is You. Here, the number of words that can no longer be used is counted and penalized. For the different words, different conditions apply as to when they count as "unusable". For instance, the connecting word *IS* is not allowed to be in any corner, while suffixes may be in all corners except the top left because only here they can no longer form a rule. Words that already form a rule are excluded from punishment. The number of words that are "out-of-plan" can be either rewarded or punished.

Suppression of passive behavior: Even though actions that lead to an already visited map state are not considered by the agent, there are some levels in which there are many auto movers. These enable the agent to stop moving for a few iterations without the map state remaining the same. Since this passive behavior is only helpful in a few situations, we added these heuristics to penalize/reward staying in the same place while the map state changes.

Min-/ Maximization of different unique rule combinations: The idea behind this heuristic was initially to enable the optimizers to encourage the creation of new rules that had not yet existed in the level in order to discover possible helpful new combinations. Counter-intuitive to our expectations, penalizing the number of new unique rules was usually preferred by the optimizers.

Ensure that not all Prefix words are stuck: This is a heuristic that issues a penalty should the agent make all prefix words unusable. Only prefix words whose objects are also in the level are considered here.

Rewarding of states with a path to victory: For this heuristic, we developed a divide and conquer algorithm that efficiently checks if there is a direct path to victory from any of the player's objects. If such a path exists, the heuristic rewards/punishes the agent, depending on the chosen weight by the optimizer. Naturally, such a situation is usually rewarded by the optimizer.

V. PARAMETER OPTIMIZATION

In our experiments, we test several optimization schemes to tune the parameters of our agent and thereby improve its performance. For this purpose, we make use of the levels being provided by the Keke AI Competition framework [12]. While the levels of the competition are unknown, we have multiple level sets available. The largest one "full-biy" consists of 184 levels most of which have been generated by the community using the mixed-initiative "Baba is Y'all" level editor [11].

We evaluated agents based on three criteria with decreasing importance, i.e., (1) the number of levels solved by the agent, (2) the average amount of iterations our agent needed to find a solution, and (3) the average computation time. During parameter optimization we used 50 randomly selected levels of the full level set for training, leaving 134 levels for our test set. During training, each algorithm was given a budget of 2000 iterations and a maximum of 2 seconds of computation time per level. Once any of these thresholds has been reached, the level is considered to be unsolved by the agent. Later during testing, we set the thresholds to 10000 iterations and 10 seconds of computation time per level. This corresponds to the settings used in the competition to make our experiments comparable with previous results. We made use of tighter time constraints during parameter optimization to allow for exploring a larger part of the parameter space and speed up the evaluation of solution candidates in general. A disadvantage of this setup is that due to the tighter time constraints, the optimizer might be inclined to remove a sub-heuristic for speeding up computation time.

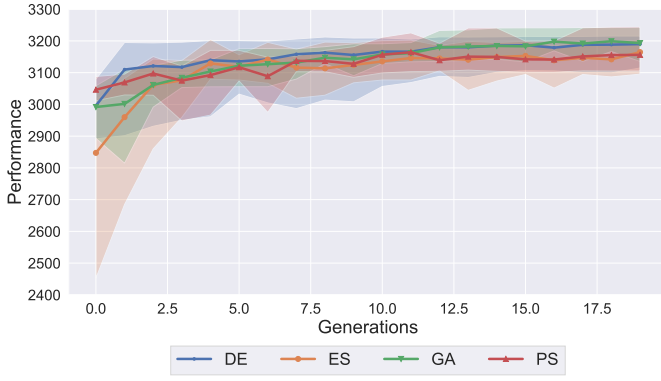
For the parameter optimization, we made use of algorithm implementations of the python package pymoo [13]. The competition framework has been extended by us to be able to send parameterized agents to the javascript-based server app. The code of our experiments and the corresponding results are provided at *link removed for double-blind review*. The following sections present details of the optimization schemes used throughout our experiments, being split into subsections for single- and multi-objective optimization algorithms.

A. Single-Objective Optimization

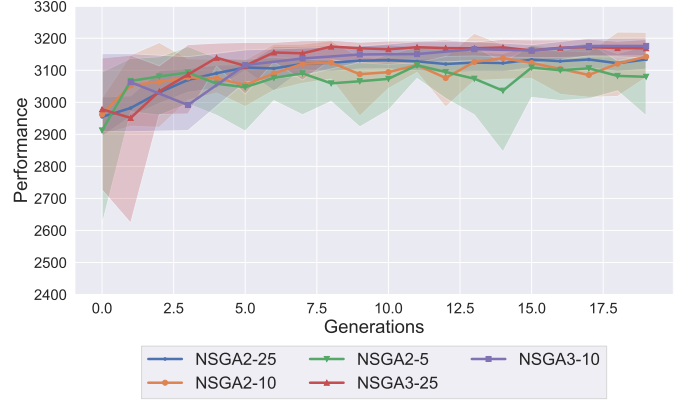
For single-objective optimization we used the algorithms differential evolution (DE) [14], evolutionary strategy (ES) [15], a genetic algorithm (GA) [15], and pattern search (PS) [16]. DE, ES, and GA had a population size of 10 and were run for 20 generations. In total each algorithm had a budget of 200 fitness evaluations. For each algorithm, we averaged the results over 5 runs using different seeds.

In our single-objective evaluation, we combined the agent's level-solving rate and the number of iterations to solve a level in a single objective. Since the ultimate goal is to achieve a high solving rate, we multiply it with the maximum number of iterations per level and add to it the remaining iterations. The latter ensures that levels are solved as fast as possible.

$$fitness = \frac{\#solved}{\#levels} \cdot max_iterations + (max_iterations - \#it)$$



(a) Single-Objective Optimization Algorithms



(b) Multi-Objective Optimization Algorithms

Fig. 2: Results of the parameter optimization. Each band shows min, mean and max values of 5 runs per optimization algorithm.

Based on the mean result per algorithm, they can be clustered into two groups. Among algorithms of the same group, the end results differ just slightly. In a comparison of all algorithms, the GA performed best by a small margin although it took longer to improve the populations' quality during the first generations. The pattern search performed worst minimal inferior to the evolutionary strategy approach. Overall all algorithms optimized without much noise which also resulted in a small variance in the end results.

B. Multi-Objective Optimization

The experiments for the multi-objective approach consisted of the NSGA-2 [17] and NSGA-3 [18], [19] algorithms. The parameters for the different algorithms were chosen so that every algorithm was trained with 200 fitness evaluations so that the results are comparable and no algorithm was preferred. To target the generality of the agent to be optimized, we formed different objectives by aggregating the performance of multiple levels of the training set to an objective. For example, a two-objective fitness function has been created by averaging the performance of the first 25 levels into the first objective while the average performance of the remaining 25 levels yields the second objective. The initialization of each algorithm differs by the number of levels used per objective. For example, 25 levels of the training set were used per objective for the algorithm instance NSGA-2-25, whereas NSGA-2-10 uses 10 levels per objective, respectively. For NSGA-3, we generated reference directions using the Das-Dennis method [20].

NSGA-3-10 had the best mean performance of all the multi-objective approaches whereas NSGA-2-5 had the worst mean performance. However, the end results of the different problem instances lie in a range of 100 points. It can be seen that the noise in the NSGA-2 problem instances is greater than in the NSGA-3 instances. Furthermore, it is shown that the noise within the NSGA-2-10 and NSGA-2-5 instances is the biggest. This confirms our expectations because the NSGA-2 algorithm has problems with more than two objectives.

C. Comparison of Results

Taking all optimization algorithms into account, the best mean performance after 200 fitness evaluations has been achieved by the genetic algorithm. It shows slightly better performance than algorithms of both the single- and multi-objective approaches. In comparison between these two, we can see that the variance of single-objective instances is smaller than the variance of the multi-objective ones.

VI. COMPARISON WITH STATE-OF-THE-ART AGENTS

For our final comparison against other Baba is You agents, we repeat the optimization process using the GA algorithm and the full budget of 10000 iterations and 10 seconds per level. Furthermore, we compare two versions of our agent. $GA_{TrainSet}$ uses the 50 training levels during optimization and runs for 100 generations, and $GA_{FullSet}$ has access to all 183 levels during optimization and is trained for 50 iterations. Both have shown to converge at the end of training.

We then compare our optimized agents to the baseline agents provided in the Keke AI framework as well as the winning entry of the Keke AI Competition 2022. The framework includes a depth-first search, a breadth-first search, and a more complex agent, called Default. The latter has been used as a solver in the context of the mixed-initiative level editor Baba is Y'all [11]. The Default agent of the framework performs an exhaustive search that uses a heuristic to prioritize its node expansion. The score consists of three functions representing different distance measurements to other objects in the level (similar to the ones in section IV-A2). The first heuristic calculates the average distance of players to the win object, the second the average distance of players to interactable words, and the last the average distance between players and objects, which have the push trait.

However, we found two major bugs in the implementation of the default agent. The first is related to the distance calculation of the three sub-heuristics. Here it is not taken into account that there are levels in which no goals or pushable objects exist in every state and this is not caught when forming

Agent	Training Levels			Test Levels			All Levels		
	Win Rate	Avg. Iter	Avg. Time	Win Rate	Avg. Iter	Avg. Time	Win Rate	Avg. Iter	Avg. Time
<i>GA_{FullSet}</i>	82.0%	898	2.13s	69.4%	1568	3.41s	72.82%	1386	3.06s
<i>Keke_{Competition}</i>	72.0%	929	3.34s	72.0%	1047	3.47s	72.00%	1015	3.44s
<i>GA_{TrainSet}</i>	82.0%	822	2.07s	67.2%	1585	3.60s	71.22%	1378	3.18s
<i>Default</i>	70.0%	1776	3.47s	58.2%	2191	4.47s	61.41%	2078	4.20s
<i>Default_{Fixed}</i>	66.0%	2166	4.02s	57.5%	2504	4.50s	59.81%	2412	4.37s
<i>BFS</i>	62.0%	4121	4.00s	54.5%	4774	4.80s	56.53%	4597	4.58s
<i>DFS</i>	58.0%	1924	4.55s	50.7%	2132	5.34s	52.68%	2075	5.13s

TABLE I: Comparison of the different agents

the average distance, resulting in a division by zero. This completely disables the heuristics for these types of levels and turns the agent into a brute-force agent. The second bug has to do with sorting the children in the queue. The standard `.sort()` Javascript function is used here, which converts the elements into strings, then compares their sequences of UTF-16 code unit values. However, this sorts a value of 11 below a value of 2, for example. This bug only affects the sorting of the queue if the map has a certain size so that the average euclidean distance can be larger than 10 in the first place. Surprisingly, the agent performed worse when fixing these bugs (cf. *Default* and *Default_{Fixed}* in Table I), which is why they could be ignored.

The best entry of the KeKe AI Competition, here called *Keke_{Competition}*, generates random solution paths. During the execution of the algorithm, these paths will be mutated as in evolutionary algorithms. The mutation operator splits the action sequence of individuals into multiple sub-strings and swaps them between individuals. The algorithm runs until a solution is found. Because of the nondeterministic nature of the algorithm, we calculated averages over 10 runs to determine the average performance of the algorithm.

Table I shows the performance of each agent on the levels of the train and the test set, as well as their aggregated performance of the full set. The best agent during our evaluation has been the *GA_{FullSet}* agent with an overall win rate of 72.82% over all 183 levels. This does not come as a surprise, since it is specifically optimized to solve the given level set. However, both the *Keke_{Competition}* agent and the *GA_{TrainSet}* agent have been close competitors. While the *Keke_{Competition}* agent achieved a win-rate of 72.00%, a difference of 1.5 levels than the best agent, the *GA_{TrainSet}* achieved a win-rate of 72.00%. The *Default* as well as the simple search-based agents achieved win rates ranging from 61.41% to 52.68%.

Moreover, the data of Table I shows that in comparison to the other agents the GA-optimized agents required the least computation time. This especially holds for levels of the training set, in which the proposed agents solved the levels in fewer iterations than other agents. Furthermore, we can see that the performance of our proposed agent degrades when applied to the test levels. Since this holds for both training schemes (FullSet and TrainSet) we assume that this difference can mostly be attributed to the varying difficulty of included levels in the train and test set.

VII. CONCLUSION

In this work, we proposed a single-heuristic search-based agent for solving Baba is You levels. The agent combined multiple heuristics into a single value. The weighting of these heuristics is optimized by several evolutionary algorithms. To speed up computation time, each sub-heuristic can be excluded entirely from the evaluation. This ensures that complex sub-heuristics are only included if they benefit the agent’s performance. Our evaluation showed slight performance improvements to existing Baba is You agents. Its strongest competitor also uses a combined heuristic function with a different search scheme. In the future, it would be interesting to see if the combination of the heuristic-function optimization using other optimization schemes could result in an even better agent.

While our agent has already shown to generalize well, there are different aspects that have room for improvement. In the following, we plan to further expand on the optimization of heuristics. In this work, evolutionary algorithms have been used to tune the weights of a given set of heuristics. Next, we plan to learn entirely new heuristics to further enhance the generality of our agent.

At the same time, we want to explore the use of our heuristics in a multiple heuristic search process such as Multiple Heuristic Greedy Best-first Search [21], Independent Multi-Heuristic A [22], portfolio-based search algorithms [23] and the Feature Space Search algorithm (FESS) [10]. Those have already been shown to perform well in their respective domains as long as the set of heuristics has been selected carefully.

REFERENCES

- [1] A. Dockhorn, S. M. Lucas, V. Volz, I. Bravi, R. D. Gaina, and D. Pérez-Liebana, “Learning local forward models on unforgiving games,” in *2019 IEEE Conference on Games (CoG)*, 2019, pp. 1–4.
- [2] D. Dor and U. Zwick, “Sokoban and other motion planning problems,” *Computational Geometry*, vol. 13, no. 4, pp. 215–228, 1999.
- [3] R. D. Gaina, J. Liu, S. M. Lucas, and D. Pérez-Liebana, “Analysis of Vanilla Rolling Horizon Evolution Parameters in General Video Game Playing,” in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 2017, vol. 10199 LNCS, pp. 418–434.
- [4] T. Virkkala, “Solving Sokoban,” Ph.D. dissertation, Ph. D. dissertation, Masters thesis, University Of Helsinki, 2011.
- [5] A. Botea, M. Müller, and J. Schaeffer, “Using abstraction for planning in Sokoban,” in *International conference on computers and games*. Springer, 2003, pp. 360–375.
- [6] A. Dockhorn and R. Kruse, *State and Action Abstraction for Search and Reinforcement Learning Algorithms*. Springer International Publishing, 2022, (to be published).

- [7] L. Xu, J. Hurtado-Gruesso, D. Jeurissen, D. P. Liebana, and A. Dockhorn, "Elastic monte carlo tree search with state abstraction for strategy game playing," *arXiv preprint arXiv:2205.15126*, 2022.
- [8] R. Pascanu, Y. Li, O. Vinyals, N. Heess, L. Buesing, S. Racanière, D. Reichert, T. Weber, D. Wierstra, and P. Battaglia, "Learning model-based planning from scratch," *arXiv preprint arXiv:1707.06170*, 2017.
- [9] M. Crippa and F. Marocchi, "Monte carlo tree search for Sokoban," Bachelor's Thesis, Politecnico Di Milano, 2018.
- [10] Y. Shoham and J. Schaeffer, "The FESS algorithm: A feature based approach to single-agent search," in *2020 IEEE Conference on Games (CoG)*, 2020, pp. 96–103.
- [11] M. Charity, I. Dave, A. Khalifa, and J. Togelius, "Baba is y'all 2.0: Design and investigation of a collaborative mixed-initiative system," *arXiv preprint arXiv:2203.02035*, 2022.
- [12] M. Charity and J. Togelius, "Keke AI competition: Solving puzzle levels in a dynamically changing mechanic space," in *2022 IEEE Conference on Games (CoG)*. IEEE, 2022, pp. 570–575.
- [13] J. Blank and K. Deb, "pymoo: Multi-objective optimization in python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.
- [14] K. Price, R. M. Storn, and J. A. Lampinen, *Differential evolution: a practical approach to global optimization*. Springer Science & Business Media, 2006.
- [15] R. Kruse, S. Mostaghim, C. Borgelt, C. Braune, and M. Steinbrecher, *Computational Intelligence*. Springer International Publishing, 2022. [Online]. Available: <https://doi.org/10.1007/978-3-030-42227-1>
- [16] R. Hooke and T. A. Jeeves, "Direct Search solution of numerical and statistical problems," *Journal of the ACM*, vol. 8, no. 2, pp. 212–229, Apr. 1961. [Online]. Available: <https://doi.org/10.1145/321062.321069>
- [17] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [18] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part i: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.
- [19] —, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part ii: Handling constraints and extending to an adaptive approach," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, p. 602–622, 2014.
- [20] I. Das and J. E. Dennis, "Normal-boundary intersection: A new method for generating the pareto surface in nonlinear multicriteria optimization problems," *SIAM Journal on Optimization*, vol. 8, no. 3, pp. 631–657, Aug. 1998. [Online]. Available: <https://doi.org/10.1137/s1052623496307510>
- [21] M. Helmert, "The fast downward planning system," *J. Artif. Int. Res.*, vol. 26, no. 1, jul 2006.
- [22] S. Aine, S. Swaminathan, V. Narayanan, V. Hwang, and M. Likhachev, "Multi-heuristic a*," *The International Journal of Robotics Research*, vol. 35, no. 1-3, pp. 224–243, Aug. 2015. [Online]. Available: <https://doi.org/10.1177/0278364915594029>
- [23] A. Dockhorn, J. Hurtado-Gruesso, D. Jeurissen, L. Xu, and D. Perez-Liebana, "Portfolio search and optimization for general strategy game-playing," in *2021 IEEE Congress on Evolutionary Computation (CEC)*, 2021, pp. 2085–2092.